

Python-Referenz

Benjamin Malicke ©

13. Dezember 2025

Inhaltsverzeichnis

1	Eingebaute Funktionen (Built-in Functions)	3
2	String-Methoden	4
3	Listen-Methoden	6
4	Dictionary-Methoden	7
5	Tuple-Methoden	8
6	Set-Methoden	8
7	Wichtige Konzepte mit Beispielen	9
7.1	Slicing	9
7.2	f-Strings	9
7.3	List Comprehension	9
7.4	Dict/Set Comprehension	9
7.5	with-Statement	10
7.6	Lambda und sorted/map/filter	10
7.7	Generator mit yield	10
7.8	Type Hints	10
7.9	Decorators (Beispiel)	10
8	Python-Kurzformen und Idiome	10
9	List Comprehensions (LC)	11
9.1	Tipp: Lesbarkeit	12
9.2	Äquivalente Formen	12
10	Dictionary Comprehensions	12
11	Set Comprehensions	13
12	Generator Expressions	14
12.1	Tipp	14
13	Python-Schlüsselwörter (Keywords)	14
14	Häufige Ausnahmen (Exceptions)	15
15	Ausgewählte Standardmodule	15
15.1	Modul: random	15
15.2	Modul: math	16
15.3	Modul: datetime (grundlegende Funktionen)	17
15.4	Modul: collections	17
15.5	Modul: itertools	18
15.6	Modul: functools	20

16 Objektorientierte Programmierung (OOP)	22
16.1 Klasse und Objekt	22
16.2 Vererbung	22
16.3 Encapsulation (private Attribute)	23
16.4 Polymorphie	23

1 Eingebaute Funktionen (Built-in Functions)

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>abs(x)</code>	int/float	Absolutwert	<code>abs(-5) # 5</code>
<code>all(iterable)</code>	bool	True, wenn alle wahr	<code>all([True, 1, "a"]) # True</code>
<code>any(iterable)</code>	bool	True, wenn mindestens eines wahr	<code>any([False, 0, ""]) # False</code>
<code>ascii(obj)</code>	str	Lesbare Darstellung (escaped)	<code>ascii("ae") # "'\xe4'"</code>
<code>bin(x)</code>	str	Binärdarstellung	<code>bin(10) # '0b1010'</code>
<code>bool(x)</code>	bool	Boolescher Wert	<code>bool(0) # False</code>
<code>chr(i)</code>	str	Zeichen zum Unicode-Codepoint	<code>chr(65) # 'A'</code>
<code>dir(obj)</code>	list	Attribute und Methoden	<code>dir([]) # Liste von List-Methoden</code>
<code>divmod(a, b)</code>	tuple	Quotient und Rest	<code>divmod(10, 3) # (3, 1)</code>
<code>enumerate(iterable)</code>	enumerate	Index + Element	<code>list(enumerate(['a', 'b'])) # [(0, 'a'), (1, 'b')]</code>
<code>eval(expr)</code>	variabel	Ausdruck auswerten	<code>eval("2 + 3") # 5</code>
<code>filter(func, iterable)</code>	filter	Gefilterte Elemente	<code>list(filter(lambda x: x>0, [-1,0,1])) # [1]</code>
<code>hex(x)</code>	str	Hexadezimal	<code>hex(255) # '0xff'</code>
<code>id(obj)</code>	int	Speicheradresse	<code>id(42) # z. B. 140735...</code>
<code>input([prompt])</code>	str	Benutzereingabe	<code>#name = input("Name: ")</code>
<code>isinstance(obj, class)</code>	bool	Typprüfung	<code>isinstance(5, int) # True</code>
<code>len(obj)</code>	int	Länge eines Objekts	<code>len("Hallo") # 5</code>

Fortsetzung auf nächster Seite...

Funktion	Rückgabotyp	Beschreibung	Beispiel
map(func, iterable)	map	Funktion anwenden	<pre>list(map(str.upper, ['a', 'b'])) # ['A', 'B']</pre>
max(iterable)	Typ des Elements	Maximalwert	<pre>max(1, 5, 3) # 5</pre>
min(iterable)	Typ des Elements	Minimalwert	<pre>min([10, 2]) # 2</pre>
oct(x)	str	Oktal	<pre>oct(8) # '0o10'</pre>
ord(c)	int	Unicode-Codepoint	<pre>ord('A') # 65</pre>
pow(x, y)	Zahl	Potenz	<pre>pow(2, 3) # 8</pre>
print(*objects)	None	Ausgabe auf Konsole	<pre>print("Hallo Welt") # Hallo Welt</pre>
range(start, stop, step)	range	Zahlenfolge	<pre>list(range(0, 10, 2)) # [0, 2, 4, 6, 8]</pre>
reversed(seq)	reversed	Umgekehrter Iterator	<pre>list(reversed([1, 2, 3])) # [3, 2, 1]</pre>
round(number)	Zahl	Runden	<pre>round(3.14159, 2) # 3.14</pre>
sorted(iterable)	list	Sortierte Liste	<pre>sorted([3, 1, 2]) # [1, 2, 3]</pre>
sum(iterable)	Zahl	Summe	<pre>sum([1, 2, 3]) # 6</pre>
type(obj)	type	Typ des Objekts	<pre>type(42) # <class 'int'></pre>
zip(*iterables)	zip	Kombiniert Iterables	<pre>list(zip([1, 2], ['a', 'b'])) # [(1, 'a'), (2, 'b')]</pre>

2 String-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
capitalize()	str	Erster Buchstabe groß	<pre>"hallo".capitalize() # " Hallo"</pre>

Methode	Rückgabotyp	Beschreibung	Beispiel
casefold()	str	Aggressives Lower	<code>"ss".casefold() # "ss"</code>
center(width)	str	Zentriert	<code>"hi".center(10) # " hi "</code>
count(sub)	int	Anzahl Vorkommen	<code>"hallo".count("l") # 2</code>
encode()	bytes	In Bytes kodieren	<code>"hallo".encode() # b'hallo'</code>
endswith(suffix)	bool	Endet mit...	<code>"hallo.txt".endswith(".txt") # True</code>
expandtabs(tabsize)	str	Tabs ersetzen	<code>"hi\tthere".expandtabs(4) # "hi there"</code>
find(sub)	int	Position (-1 wenn nicht)	<code>"hallo".find("l") # 2</code>
format(*args)	str	String formatieren	<code>"Hallo {}".format("Max") # "Hallo Max"</code>
index(sub)	int	Wie find(), aber Exception	<code>"hallo".index("l") # 2</code>
isalnum()	bool	Alphanumerisch	<code>"a1".isalnum() # True</code>
isalpha()	bool	Nur Buchstaben	<code>"abc".isalpha() # True</code>
isdigit()	bool	Nur Ziffern	<code>"123".isdigit() # True</code>
islower()	bool	Kleinbuchstaben	<code>"hallo".islower() # True</code>
isupper()	bool	Großbuchstaben	<code>"HALLO".isupper() # True</code>
join(iterable)	str	Verbindet	<code>"-".join(["a", "b"]) # "a-b"</code>
ljust(width)	str	Linksbündig	<code>"hi".ljust(10) # "hi "</code>
lower()	str	In Kleinbuchstaben	<code>"HALLO".lower() # "hallo"</code>
lstrip()	str	Links Whitespace entfernen	<code>" hi".lstrip() # "hi"</code>

Methode	Rückgabotyp	Beschreibung	Beispiel
partition(sep)	tuple	Teilt in 3 Teile	<pre>"hallo welt".partition(" ") # ('hallo', ' ', ' welt')</pre>
replace(old, new)	str	Ersetzt	<pre>"hallo".replace("l", "x") # "haaxo"</pre>
rfind(sub)	int	Sucht von rechts	<pre>"hallo".rfind("l") # 3</pre>
rjust(width)	str	Rechtsbündig	<pre>"hi".rjust(10) # " hi"</pre>
rstrip()	str	Rechts Whitespace entfernen	<pre>"hi ".rstrip() # "hi"</pre>
split(sep)	list	Teilt String	<pre>"a b c".split() # ['a', 'b ', 'c']</pre>
splitlines()	list	Zeilenumbrüche	<pre>"a\nb".splitlines() # ['a', 'b']</pre>
startswith(prefix)	bool	Prüft Anfang	<pre>"hallo".startswith("ha") # True</pre>
strip()	str	Entfernt Whitespace	<pre>" hi ".strip() # "hi"</pre>
swapcase()	str	Groß/Klein tauschen	<pre>"Hallo".swapcase() # "hALLO"</pre>
title()	str	Titel-Format	<pre>"hallo welt".title() # " Hallo Welt"</pre>
upper()	str	In Großbuchstaben	<pre>"hallo".upper() # "HALLO"</pre>
zfill(width)	str	Mit Nullen auffüllen	<pre>"42".zfill(5) # "00042"</pre>

3 Listen-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
append(x)	None	Am Ende hinzufügen	<pre>lst = [1]; lst.append(2) # [1,2]</pre>
clear()	None	Alle entfernen	<pre>lst.clear() # []</pre>

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>copy()</code>	<code>list</code>	Flache Kopie	<code>lst.copy()</code> # neue Liste
<code>count(x)</code>	<code>int</code>	Anzahl Vorkommen	<code>[1,2,1].count(1)</code> # 2
<code>extend(iter)</code>	<code>None</code>	Erweitern	<code>lst.extend([3,4])</code> # <code>[1,2,3,4]</code>
<code>index(x)</code>	<code>int</code>	Position (Exception wenn nicht)	<code>[1,2,3].index(2)</code> # 1
<code>insert(i, x)</code>	<code>None</code>	An Position einfügen	<code>lst.insert(0, 0)</code> # <code>[0,1,2,3,4]</code>
<code>pop([i])</code>	Element	Entfernen und zurückgeben	<code>lst.pop()</code> # 4
<code>remove(x)</code>	<code>None</code>	Erstes Vorkommen entfernen	<code>lst.remove(1)</code> # entfernt 1
<code>reverse()</code>	<code>None</code>	Umkehren	<code>lst.reverse()</code> # umgedreht
<code>sort()</code>	<code>None</code>	In-place sortieren	<code>lst.sort(reverse=True)</code> # <code>absteigend</code>

4 Dictionary-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>clear()</code>	<code>None</code>	Alle entfernen	<code>d.clear()</code> # {}
<code>copy()</code>	<code>dict</code>	Flache Kopie	<code>d.copy()</code> # neue Dict
<code>fromkeys(keys, val)</code>	<code>dict</code>	Dict aus Keys erstellen	<code>dict.fromkeys(['a','b'], 0)</code> # {'a':0, 'b':0}
<code>get(key[, default])</code>	Wert oder default	Wert holen	<code>d.get("b", 0)</code> # 0
<code>items()</code>	<code>dict_items</code>	View auf Paare	<code>list(d.items())</code> # [('a', 1)]
<code>keys()</code>	<code>dict_keys</code>	View auf Keys	<code>list(d.keys())</code> # ['a']
<code>pop(key)</code>	Wert	Entfernen und zurückgeben	<code>d.pop("a")</code> # 1

Methode	Rückgabotyp	Beschreibung	Beispiel
popitem()	Tuple	Letztes Paar entfernen	<code>d.popitem()</code> # ('b', 2)
setdefault(key, default)	Wert	Wert holen oder setzen	<code>d.setdefault("c", 3)</code> # 3
update(other)	None	Aktualisieren	<code>d.update({"b":2})</code> # {"a":1, "b":2}
values()	dict_values	View auf Werte	<code>list(d.values())</code> # [1]

5 Tuple-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
count(x)	int	Anzahl Vorkommen	<code>(1,2,1).count(1)</code> # 2
index(x)	int	Position	<code>(1,2,3).index(2)</code> # 1

6 Set-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
add(x)	None	Hinzufügen	<code>s = {1}; s.add(2)</code> # {1,2}
clear()	None	Alle entfernen	<code>s.clear()</code> # set()
copy()	set	Kopie	<code>s.copy()</code> # neue Set
difference(other)	set	Differenz	<code>{1,2}.difference({2})</code> # {1}
discard(x)	None	Entfernen (kein Fehler)	<code>s.discard(99)</code> # nichts passiert
intersection(other)	set	Schnittmenge	<code>{1,2}.intersection({2,3})</code> # {2}
isdisjoint(other)	bool	Keine Schnittmenge	<code>{1,2}.isdisjoint({3})</code> # True
issubset(other)	bool	Teilmenge	<code>{1,2}.issubset({1,2,3})</code> # True

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>issuperset(other)</code>	<code>bool</code>	Obermenge	<code>{1,2,3}.issuperset({1,2})</code> # <i>True</i>
<code>pop()</code>	Element	Beliebiges entfernen	<code>s.pop()</code> # z. B. 1
<code>remove(x)</code>	<code>None</code>	Entfernen (mit Fehler)	<code>s.remove(1)</code> # <i>entfernt 1</i>
<code>symmetric_difference(other)</code>	<code>set</code>	Symmetrische Differenz	<code>{1,2}.symmetric_difference({2,3})</code> # <i>{1,3}</i>
<code>union(other)</code>	<code>set</code>	Vereinigung	<code>{1,2}.union({3})</code> # <i>{1,2,3}</i>
<code>update(other)</code>	<code>None</code>	Vereinigung in-place	<code>s.update({4})</code> # <i>fuegt 4 hinzu</i>

7 Wichtige Konzepte mit Beispielen

7.1 Slicing

```

1 lst = [0,1,2,3,4]
2 lst[1:4] # [1,2,3]
3 lst[::-1] # [4,3,2,1,0]
4 lst[-2:] # [3,4]
5 s = "Python"
6 s[1:4] # "yth"
7 s[::-1] # "nohtyP"

```

7.2 f-Strings

```

1 name = "Anna"
2 alter = 25
3 f"Hallo {name}, du bist {alter*2} in doppelt."
4 # "Hallo Anna, du bist 50 in doppelt."
5 f"{alter * 2 = }" # "alter * 2 = 50"

```

7.3 List Comprehension

```

1 [x**2 for x in range(5) if x % 2 == 0]
2 # [0, 4, 16]

```

7.4 Dict/Set Comprehension

```

1 {x: x**2 for x in range(5)} # {0:0, 1:1, ...}
2 {x for x in range(5) if x % 2 == 0} # {0,2,4}

```

7.5 with-Statement

```
1 with open("datei.txt", "w") as f:
2     f.write("Hallo")
3 # Datei automatisch geschlossen
```

7.6 Lambda und sorted/map/filter

```
1 sorted(["apfel", "Banane"], key=lambda s: s.lower())
2 # ['apfel', 'Banane']
3
4 list(map(lambda x: x**2, [1,2,3])) # [1,4,9]
```

7.7 Generator mit yield

```
1 def gen():
2     yield 1
3     yield 2
4
5 list(gen()) # [1,2]
```

7.8 Type Hints

```
1 def add(a: int, b: int) -> int:
2     return a + b
```

7.9 Decorators (Beispiel)

```
1 def timer(func):
2     import time
3     def wrapper(*args, **kwargs):
4         start = time.time()
5         result = func(*args, **kwargs)
6         print(f"Zeit: {time.time() - start}s")
7         return result
8     return wrapper
9
10 @timer
11 def langsam():
12     time.sleep(1)
13
14 langsam()
```

8 Python-Kurzformen und Idiome

Konstrukt	Äquivalent	Beschreibung	Beispiel
Ternärer Operator	if-else	Bedingter Ausdruck	<pre>1 x = 10 2 a if x > 5 else b 3 # gleichbedeutend mit: 4 # a if x > 5 else b</pre>
a if bedingung else b	if-Anweisung	Wert je nach Bedingung	<pre>1 status = "positiv" if x > 0 else "nicht positiv"</pre>

Konstrukt	Äquivalent	Beschreibung	Beispiel
Multiple Zuweisung	Einzelzuweisungen	Gleichzeitiges Zuweisen	<pre> 1 a, b = 1, 2 2 a, b = b, a # tauschen ohne temp </pre>
Unpacking	Indexzugriff	Entpacken von Iterables	<pre> 1 head, *tail = [1,2,3,4] 2 # head=1, tail=[2,3,4] 3 4 x, y, *_ = (10, 20, 30, 40) 5 # x=10, y=20, _=[30,40] </pre>
Walrus-Operator (:=)	Separate Zuweisung	Zuweisen innerhalb Ausdruck (ab 3.8)	<pre> 1 if (n := len(data)) > 10: 2 print(f"Liste hat {n} Elemente") </pre>
Chained Comparisons	Mehrere Vergleiche	Lesbare Vergleichsketten	<pre> 1 if 0 <= x <= 100: 2 print("x im Bereich 0-100") 3 # gleich: if x >= 0 and x <= 100 </pre>
or-Default (CL-style)	if-Abfrage	Häufiges Idiom für Default-Werte	<pre> 1 name = user_input or " Anonym" 2 # wenn user_input falsy -> "Anonym" </pre>
and-Kurzschluss	if	Bedingte Ausführung	<pre> 1 data and process(data) 2 # process nur, wenn data truthy </pre>
List/Dict/Set aus Iterable	Schleife + append	Comprehension (siehe auch früher)	<pre> 1 quadrate = [x*x for x in range(10)] 2 woerterbuch = {k: v for k,v in paare} </pre>
else-Klausel bei Schleifen	–	Wird ausgeführt, wenn keine break	<pre> 1 for i in range(10): 2 if i == 15: 3 break 4 else: 5 print("Nicht abgebrochen") 6 # Wird gedruckt </pre>

9 List Comprehensions (LC)

List Comprehensions sind eine kompakte und „pythonische“ Möglichkeit, neue Listen zu erzeugen. Sie ersetzen oft eine for-Schleife mit `append()` und sind schneller sowie besser lesbar.

Form	Äquivalent	Beschreibung	Beispiel
[expr for item in iter]	Schleife + append	Grundform	<pre> 1 [x for x in range(5)] 2 # [0, 1, 2, 3, 4] </pre>

Form	Äquivalent	Beschreibung	Beispiel
[expr for item in iter if bedingung]	Schleife + if	Mit Filter	<pre> 1 [x for x in range(10) if x % 2 == 0] 2 # [0, 2, 4, 6, 8]</pre>
Verschachtelt	Doppelte Schleife	Reihenfolge wie verschachtelte for	<pre> 1 [(x, y) for x in [1,2] for y in ['a','b']] 2 # [(1,'a'), (1,'b'), (2,'a'), (2,'b')]</pre>
Mit Ausdruck	Transformation	Beliebiger Ausdruck vorne	<pre> 1 [x**2 for x in range(6)] 2 # [0, 1, 4, 9, 16, 25]</pre>
Mit Funktion	map-Äquivalent	Funktion auf Elemente anwenden	<pre> 1 [s.upper() for s in ["hallo", "welt"]] 2 # ['HALLO', 'WELT']</pre>
Bedingter Ausdruck	if-else	Ternärer Operator innerhalb LC	<pre> 1 ["gerade" if x % 2 == 0 else "ungerade" 2 for x in range(5)] 3 # ['gerade', 'ungerade', 'gerade', ...]</pre>
Flattening	concat	Liste von Listen einebnen	<pre> 1 [y for x in [[1,2],[3,4]] for y in x] 2 # [1, 2, 3, 4]</pre>
Primzahlen (klassisch)	Schleife + all()	Komplexeres Beispiel	<pre> 1 from math import sqrt 2 [n for n in range(2, 30) 3 if all(n % i != 0 4 for i in range(2, int(sqrt(n))+1))] 5 # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]</pre>
Nur Buchstaben	filter	Zeichen filtern	<pre> 1 [c for c in "abc123def" if c.isalpha()] 2 # ['a', 'b', 'c', 'd', 'e', 'f']</pre>

9.1 Tipp: Lesbarkeit

List Comprehensions sind genial, aber bei sehr komplexen Ausdrücken (z. B. tief verschachtelt oder lange Bedingungen) ist eine klassische for-Schleife oft klarer. Die Faustregel lautet: **Eine LC sollte in eine Zeile passen und sofort verständlich sein.**

9.2 Äquivalente Formen

Es gibt auch:

- {key: value for ...} → Dictionary Comprehension
- {expr for ...} → Set Comprehension
- (expr for ...) → Generator Expression (speichersparend, kein [])

10 Dictionary Comprehensions

Dictionary Comprehensions erzeugen ein neues Dictionary in kompakter Form. Syntax: {key: value for ...}

Form	Äquivalent	Beschreibung	Beispiel
<code>{k: v for k, v in iter}</code>	Schleife + Zuweisung	Grundform	<pre> 1 {w: w.upper() for w in "abc"} 2 # {'a': 'A', 'b': 'B', 'c': 'C'}</pre>
Mit Index	enumerate	Schlüssel als Laufindex	<pre> 1 {idx: c for idx, c in enumerate("hallo")} 2 # {0: 'h', 1: 'a', 2: 'l', 3: 'l', 4: 'o'}</pre>
Mit Filter	if-Bedingung	Nur bestimmte Einträge	<pre> 1 {x: x**2 for x in range(10) if x % 2 == 0} 2 # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}</pre>
Werte umkehren	swap key/value	Keys und Values tauschen	<pre> 1 d = {'a':1, 'b':2, 'c':1} 2 {v: k for k, v in d.items()} 3 # {1: 'c', 2: 'b'} # letzter Key gewinnt</pre>
Zählen (Alternative zu Counter)	manuell	Häufigkeiten zählen	<pre> 1 s = "abracadabra" 2 {c: s.count(c) for c in set(s)} 3 # {'a':5, 'b':3, 'r':2, 'c':1, 'd':1}</pre>
Default-Wert bei Inversion	or-Default	Vermeidet KeyError	<pre> 1 d = {'a':1, 'b':2} 2 {k: v*10 for k, v in d.items()} 3 # {'a':10, 'b':20}</pre>

11 Set Comprehensions

Set Comprehensions erzeugen ein neues Set (ohne Duplikate, ungeordnet). Syntax: `{expr for ...}`

Form	Äquivalent	Beschreibung	Beispiel
<code>{expr for item in iter}</code>	Schleife + add	Grundform	<pre> 1 {x**2 for x in [1,2,2,3,3]} 2 # {1, 4, 9} # Duplikate entfernt</pre>
Mit Filter	if-Bedingung	Nur bestimmte Elemente	<pre> 1 {c for c in "abracadabra" if c in "abc"} 2 # {'a', 'b', 'c'}</pre>
Einzigartige Längen	len()	Längen von Wörtern	<pre> 1 {len(w) for w in ["a", "bb", "ccc", "bb"]} 2 # {1, 2, 3}</pre>
Buchstaben eines Texts	set()	Alle verwendeten Zeichen	<pre> 1 {c.upper() for c in "Hallo Welt!" if c. isalpha()} 2 # {'A', 'E', 'H', 'L', 'O', 'T', 'W'}</pre>
Primzahlen als Set	LC → Set	Schnelle Umwandlung	<pre> 1 from math import sqrt 2 {n for n in range(2, 20) 3 if all(n % i != 0 for i in range 4 (2, int(sqrt(n))+1))}</pre>

12 Generator Expressions

Generator Expressions erzeugen einen ****Generator**** (lazy, speichersparend) statt einer Liste. Syntax: (expr for ...) – runde Klammern statt eckig.

Sie werden oft an Funktionen übergeben, die einen Iterator erwarten (sum(), max(), list(), any() usw.).

Form	Äquivalent	Beschreibung	Beispiel
(expr for item in iter)	LC in runden Klammern	Lazy-Version	<pre> 1 g = (x**2 for x in range(5)) 2 list(g) # [0, 1, 4, 9, 16]</pre>
Mit sum()/max()/any()	direkt	Keine Liste nötig	<pre> 1 sum(x**2 for x in range(1000)) 2 # 332833500 # sehr speichersparend</pre>
Große Datei zeilenweise	open()	Nur eine Zeile im Speicher	<pre> 1 sum(1 for line in open("datei.txt") if 2 line.strip()) # Anzahl nicht-leerer Zeilen</pre>
Mit frühem Stopp	short-circuit	Prüft nur bis erste True	<pre> 1 any(c.isdigit() for c in "abcdefghi") 2 # True - stoppt bei '1' (nach 4 Zeichen)</pre>
Unendliche Folge	count/repeat	Z. B. mit itertools	<pre> 1 from itertools import count, islice 2 list(islice((x**2 for x in count()), 3 10)) # [0, 1, 4, 9, ..., 81]</pre>
Vergleich Speicher- verbrauch	list vs. gen	Praktischer Unterschied	<pre> 1 import sys 2 lst = [x*x for x in range(1000000)] 3 gen = (x*x for x in range(1000000)) 4 sys.getsizeof(lst) # >> mehrere MB 5 sys.getsizeof(gen) # ~ 200 Bytes</pre>

12.1 Tipp

Verwende Generator Expressions immer dann, wenn:

- du nur einmal über die Daten iterierst,
- die Datenmenge groß ist,
- du das Ergebnis direkt an eine Funktion übergibst (sum, max, all, list, set usw.).

13 Python-Schlüsselwörter (Keywords)

Schlüsselwort	Kategorie	Beschreibung
False / True / None	Konstanten	Boolesche Werte und Nullwert
and / or / not	Logik	Logische Operatoren
if / elif / else	Bedingung	Verzweigungen
for / while	Schleifen	Iteration
break / continue / pass	Schleifensteuerung	Steuerung von Schleifen / Platzhalter
def	Funktion	Funktionsdefinition
lambda	Funktion	Anonyme Funktion
return	Funktion	Rückgabewert
yield	Generator	Generator-Funktion

Schlüsselwort	Kategorie	Beschreibung
class	OOP	Klassendefinition
global / nonlocal	Scope	Variablenscope beeinflussen
import / from / as	Module	Modulimport
try / except / finally / raise	Fehler	Exception-Handling
with / as	Kontext	Context Manager
assert	Debugging	Bedingung prüfen (für Tests)
del	Löschung	Objekt/Attribut löschen
in / is	Operatoren	Mitgliedschaft und Identität
async / await	Asynchron	Asynchrone Programmierung
match / case	Pattern Matching	Strukturelle Musterabgleich (ab Python 3.10)

14 Häufige Ausnahmen (Exceptions)

Ausnahme	Erbt von	Beschreibung / Typischer Fall
Exception	–	Basisklasse aller Exceptions
ArithmeticError	Exception	Arithmetische Fehler (z. B. Overflow)
ZeroDivisionError	ArithmeticError	Division durch Null
AssertionError	Exception	Fehlgeschlagenes <code>assert</code>
AttributeError	Exception	Zugriff auf nicht existierendes Attribut
IndexError	LookupError	Ungültiger Listen-/Tuple-Index
KeyError	LookupError	Fehlender Dictionary-Key
NameError	Exception	Unbekannte Variable/Funktion
TypeError	Exception	Falscher Datentyp (z. B. <code>ä" + 1</code>)
ValueError	Exception	Falscher Wert (z. B. <code>int(äbc)</code>)
ImportError	Exception	Modul nicht gefunden
ModuleNotFoundError		
FileNotFoundError	OSError	Datei nicht gefunden
KeyboardInterrupt	BaseException	Programm durch Strg+C abgebrochen
SyntaxError	Exception	Ungültiger Python-Code
IndentationError	SyntaxError	Falsche Einrückung
OverflowError	ArithmeticError	Numerischer Überlauf
MemoryError	Exception	Zu wenig Speicher
NotImplementedError	RuntimeError	Methode noch nicht implementiert
RuntimeError	Exception	Allgemeiner Laufzeitfehler

15 Ausgewählte Standardmodule

15.1 Modul: random

Funktion	Rückgabetyt	Beschreibung	Beispiel
random()	float	Zufallszahl 0.0 bis <1.0	<pre>1 import random 2 random.random() # z. B. 0.734</pre>
uniform(a, b)	float	Zufallszahl zwischen a und b	<pre>1 random.uniform(1, 10) # z. B. 7.32</pre>
randint(a, b)	int	Ganzzahl inkl. a und b	<pre>1 random.randint(1, 6) # Wurfel: 1-6</pre>
randrange(stop)			

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>randrange(start, stop[, step])</code>	int	Wie <code>range()</code> , aber zufällig	<pre>1 random.randrange(0, 10, 2) # gerade Zahl 0-8</pre>
<code>choice(seq)</code>	Element	Zufälliges Element	<pre>1 random.choice(['a', 'b', 'c']) # z. B. 'b'</pre>
<code>choices(seq, k=n)</code>	list	n Elemente mit Zurücklegen	<pre>1 random.choices(['rot', 'blau'], k=3) 2 # z. B. ['rot', 'rot', 'blau']</pre>
<code>sample(seq, k=n)</code>	list	n einzigartige Elemente	<pre>1 random.sample(range(100), 5) # 5 Lottozahlen</pre>
<code>shuffle(seq)</code>	None	Mischt in-place	<pre>1 lst = [1,2,3]; random.shuffle(lst) # lst veraendert</pre>
<code>seed(a)</code>	None	Seed setzen (reproduzierbar)	<pre>1 random.seed(42) # immer gleiche Folge</pre>

15.2 Modul: math

Funktion/Konstante	Rückgabotyp	Beschreibung	Beispiel
<code>pi</code> / <code>e</code> / <code>tau</code>	float	Kreiszahl, Euler, 2π	<pre>1 import math 2 math.pi # 3.141592653589793</pre>
<code>sqrt(x)</code>	float	Quadratwurzel	<pre>1 math.sqrt(16) # 4.0</pre>
<code>pow(x, y)</code>	float	x hoch y (wie <code>**</code> , aber float)	<pre>1 math.pow(2, 3) # 8.0</pre>
<code>floor(x)</code> / <code>ceil(x)</code>	int	Abrunden / Aufrunden	<pre>1 math.floor(3.7) # 3 2 math.ceil(3.2) # 4</pre>
<code>trunc(x)</code>	int	Richtung 0 abschneiden	<pre>1 math.trunc(-3.7) # -3</pre>
<code>fabs(x)</code>	float	Absolutwert (float)	<pre>1 math.fabs(-5) # 5.0</pre>
<code>sin(x)</code> / <code>cos(x)</code> / <code>tan(x)</code>	float	Trigonometrie (Radiant)	<pre>1 math.sin(math.pi/2) # 1.0</pre>
<code>asin(x)</code> / <code>acos(x)</code>	float	Arkusfunktionen	<pre>1 math.asin(1) # pi/2</pre>

Funktion/Konstante	Rückgabotyp	Beschreibung	Beispiel
degrees(x) / radians(x)	float	Umrechnung Grad <-> Radiant	<pre>math.degrees(math.pi) # 180.0</pre>
log(x[, base])	float	Logarithmus	<pre>math.log(100, 10) # 2.0</pre>
log10(x) / log2(x)	float	Dekadisch / Binär	<pre>math.log10(100) # 2.0</pre>
factorial(n)	int	Fakultät	<pre>math.factorial(5) # 120</pre>
gcd(*integers)	int	Größter gemeinsamer Teiler	<pre>math.gcd(12, 18) # 6</pre>
comb(n, k) / perm(n, k)	int	Binomialkoeffizient / Permutation	<pre>math.comb(5, 2) # 10</pre>

15.3 Modul: datetime (grundlegende Funktionen)

Funktion/Eigenschaft	Rückgabotyp	Beschreibung	Beispiel
datetime.now()	datetime	Aktuelles Datum+Uhrzeit	<pre>1 from datetime import datetime 2 datetime.now() # 2025-12-13 14:30:...</pre>
datetime.today() date.today()	datetime date	Wie now() Nur Datum	<pre>1 from datetime import date 2 date.today() # 2025-12-13</pre>
datetime.strptime(str, fmt)	datetime	String → datetime	<pre>1 datetime.strptime("2025-12-13", "%Y-%m-%d")</pre>
dt.strftime(fmt)	str	datetime → String	<pre>1 datetime.now().strftime("%d.%m.%Y") 2 # "13.12.2025"</pre>
dt.year / dt.month / dt.day	int	Bestandteile	
dt.hour / dt.minute / dt.second	int	Uhrzeit	
timedelta(days=...)	timedelta	Zeitdifferenz	<pre>1 from datetime import timedelta 2 datetime.now() + timedelta(days=7) 3 # in einer Woche</pre>

15.4 Modul: collections

Wichtige Klassen und Funktionen aus dem collections-Modul (erweiterte Datentypen).

Klasse/Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>namedtuple(typename, field_names)</code>	Typ	Tuple mit benannten Feldern	<pre> 1 from collections import namedtuple 2 Point = namedtuple('Point', ['x', 'y']) 3 p = Point(1, 2) 4 p.x # 1 5 p[0] # 1 (wie normales Tuple) </pre>
<code>deque([iterable])</code>	<code>deque</code>	Doppel-endige Queue (schnelles append/pop links/rechts)	<pre> 1 from collections import deque 2 d = deque([1,2,3]) 3 d.appendleft(0) # deque ([0,1,2,3]) 4 d.popleft() # 0 </pre>
<code>Counter([iterable])</code>	<code>Counter</code>	Zählt Häufigkeiten (subclass von dict)	<pre> 1 from collections import Counter 2 c = Counter("abracadabra") 3 c['a'] # 5 4 c.most_common(2) # [('a',5) , ('b',2)] </pre>
<code>OrderedDict()</code>	<code>OrderedDict</code>	Dictionary mit Einfügereihenfolge (bis Python 3.7 nötig)	<pre> 1 from collections import OrderedDict 2 od = OrderedDict([('a',1), ('b',2)]) 3 list(od.keys()) # ['a', 'b' '] </pre>
<code>defaultdict(default_fac)</code>	<code>defaultdict</code>	Dict mit Default-Wert bei fehlendem Key	<pre> 1 from collections import defaultdict 2 dd = defaultdict(int) 3 dd['a'] += 1 # dd['a'] == 1 (automatisch 0 bei erstem Zugriff) </pre>
<code>ChainMap(*maps)</code>	<code>ChainMap</code>	Logische Verkettung mehrerer Dictionaries	<pre> 1 from collections import ChainMap 2 a = {'x':1}; b = {'y':2} 3 cm = ChainMap(a, b) 4 cm['x'] # 1 5 cm['y'] # 2 </pre>

15.5 Modul: itertools

Leistungsstarke Iterator-Werkzeuge für effiziente Schleifen und Kombinatorik.

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>count(start=0, step=1)</code>	Iterator	Unendliche Zahlenfolge	<pre> 1 from itertools import count 2 list(next(islice(count(10), 5))) 3 # [10,11,12,13,14] </pre>

Funktion	Rückgabotyp	Beschreibung	Beispiel
cycle(iterable)	Iterator	Endlos wiederholen	<pre> 1 from itertools import cycle , islice 2 list(islice(cycle('AB'), 7)) 3 # ['A', 'B', 'A', 'B', 'A', 'B , 'A'] </pre>
repeat(object[, times])	Iterator	Objekt wiederholen	<pre> 1 list(repeat(10, 3)) # [10, 10, 10] </pre>
chain(*iterables)	Iterator	Iterables verkettet	<pre> 1 from itertools import chain 2 list(chain([1, 2], 'ab')) # [1, 2, 'a', 'b'] </pre>
compress(data, selectors)	Iterator	Gefiltert nach True-Werten	<pre> 1 list(compress('ABCDEF', [1, 0, 1, 0, 1, 1])) 2 # ['A', 'C', 'E', 'F'] </pre>
dropwhile(predicate, iterable)	Iterator	Überspringt, solange Bedingung wahr	<pre> 1 from itertools import dropwhile 2 list(dropwhile(lambda x: x < 5, [1, 4, 6, 4, 1])) 3 # [6, 4, 1] </pre>
takewhile(predicate, iterable)	Iterator	Nimmt, solange Bedingung wahr	<pre> 1 list(takewhile(lambda x: x < 5, [1, 4, 6, 4, 1])) 2 # [1, 4] </pre>
filterfalse(predicate, iterable)	Iterator	Elemente, für die predicate False	<pre> 1 from itertools import filterfalse 2 list(filterfalse(lambda x: x%2, range(10))) 3 # [0, 2, 4, 6, 8] </pre>
groupby(iterable, key=None)	Iterator	Gruppiert nach Schlüssel	<pre> 1 from itertools import groupby 2 [k for k, g in groupby(' AAAABBBCC')] 3 # ['A', 'B', 'C'] </pre>
islice(iterable, start, stop[, step])	Iterator	Slice eines Iterators	<pre> 1 from itertools import islice 2 list(islice('ABCDEFGH', 2, None)) 3 # ['C', 'D', 'E', 'F', 'G'] </pre>
starmap(function, iterable)	Iterator	map mit unpack	<pre> 1 from itertools import starmap 2 list(starmap(pow, [(2, 5), (3, 2)])) 3 # [32, 9] </pre>

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>tee(iterable, n=2)</code>	n-Tupel	Mehrere unabhängige Iteratoren	<pre> 1 from itertools import tee 2 a, b = tee([1,2,3], 2) 3 list(a) # [1,2,3] 4 list(b) # [1,2,3]</pre>
<code>product(*iterables, repeat=1)</code>	Iterator	Kartesisches Produkt	<pre> 1 from itertools import product 2 list(product('AB', [1,2])) 3 # [('A', 1), ('A', 2), ('B', 1), ('B', 2)]</pre>
<code>permutations(iterable, r=None)</code>	Iterator	Permutationen	<pre> 1 list(permutations('AB', 2)) 2 # [('A', 'B'), ('B', 'A')]</pre>
<code>combinations(iterable, r)</code>	Iterator	Kombinationen (ohne Reihenfolge)	<pre> 1 list(combinations('ABCD', 2 2)) 3 # [('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D'), ('C', 'D')]</pre>
<code>combinations_with_replacement(iterable, r)</code>	Iterator	Kombinationen mit Wiederholung	<pre> 1 list(2 combinations_with_replacement('AB', 2)) 3 # [('A', 'A'), ('A', 'B'), ('B', 'B')]</pre>
<code>accumulate(iterable[, func])</code>	Iterator	Kumulative Berechnung	<pre> 1 from itertools import accumulate 2 import operator 3 list(accumulate([1,2,3,4], operator.mul)) 4 # [1, 2, 6, 24]</pre>

15.6 Modul: functools

Nützliche Higher-Order-Funktionen und Dekoratoren für funktionale Programmierung.

Funktion/Dekorator	Rückgabotyp	Beschreibung	Beispiel
<code>partial(func, *args, **kw)</code>	partial	Funktion mit vorgegebenen Argumenten	<pre> 1 from functools import partial 2 hex8 = partial(int, base =16) 3 hex8('ff') # 255</pre>

Funktion/Dekorator	Rückgabotyp	Beschreibung	Beispiel
cache / lru_cache(maxsize=128)	Dekorator	Memoization (Ergebnisse cachen)	<pre> 1 from functools import lru_cache 2 3 @lru_cache(maxsize=None) 4 def fib(n): 5 if n < 2: return n 6 return fib(n-1) + fib(n-2) 7 8 fib(100) # sehr schnell dank Cache </pre>
cached_property	Dekorator	Property, die nur einmal berechnet wird	<pre> 1 from functools import cached_property 2 3 class Kreis: 4 def __init__(self, r): 5 self.r = r 6 7 @cached_property 8 def flaeche(self): 9 return 3.14159 * self.r ** 2 10 11 k = Kreis(5) 12 k.flaeche # berechnet einmal, dann gecacht </pre>
reduce(function, iterable[, initial])	Wert	Kumulative Anwendung einer Funktion	<pre> 1 from functools import reduce 2 import operator 3 reduce(operator.add, [1,2,3,4]) # 10 </pre>
singledispatch	Dekorator	Funktionsüberladung nach Typ (Generic Function)	<pre> 1 from functools import singledispatch 2 3 @singledispatch 4 def verarbeite(obj): 5 print("Standard:", obj) 6 7 @verarbeite.register 8 def _(obj: int): 9 print("Integer:", obj) 10 11 @verarbeite.register 12 def _(obj: str): 13 print("String:", obj) 14 15 verarbeite(42) # Integer: 42 16 verarbeite("Hallo") # String: Hallo </pre>

Funktion/Dekorator	Rückgabotyp	Beschreibung	Beispiel
singledispatchmethod	Dekorator	Wie singledispatch, aber für Methoden	<pre> 1 from functools import singledispatchmethod 2 3 class Prozessor: 4 @singledispatchmethod 5 def verarbeite(self, arg): 6 print("Standard:", arg) 7 8 @verarbeite.register 9 def _(self, arg: int): 10 print("Integer:", arg) </pre>
cmp_to_key(func)	key	Vergleichsfunktion (altes cmp) → key-Funktion für sort	<pre> 1 from functools import cmp_to_key 2 3 def cmp(a, b): 4 return (a > b) - (a < b) 5 6 sorted([3,1,2], key= cmp_to_key(cmp)) 7 # [1, 2, 3] </pre>
wraps	Dekorator	Hilfsdekorator, um Metadaten der Originalfunktion zu erhalten	<pre> 1 from functools import wraps 2 3 def mein_deko(f): 4 @wraps(f) 5 def wrapper(*args, **kw): 6 print("Vor Aufruf") 7 return f(*args, **kw) 8 return wrapper 9 10 @mein_deko 11 def hallo(): 12 """Sagt Hallo""" 13 print("Hallo") 14 15 hallo.__name__ # 'hallo' (dank wraps) 16 hallo.__doc__ # 'Sagt Hallo ' </pre>

16 Objektorientierte Programmierung (OOP)

16.1 Klasse und Objekt

```

1 class Person:
2     def __init__(self, name, alter):
3         self.name = name
4         self.alter = alter
5
6     def vorstellen(self):
7         print(f"Hallo, ich bin {self.name} und {self.alter} Jahre alt.")
8
9 p = Person("Max", 30)
10 p.vorstellen() # Hallo, ich bin Max und 30 Jahre alt.

```

16.2 Vererbung

```

1 class Student(Person):
2     def __init__(self, name, alter, matrikel):
3         super().__init__(name, alter)
4         self.matrikel = matrikel
5
6     def vorstellen(self):
7         super().vorstellen()
8         print(f"Meine Matrikelnummer ist {self.matrikel}.")
9
10 s = Student("Anna", 22, "12345")
11 s.vorstellen()

```

16.3 Encapsulation (private Attribute)

```

1 class Bankkonto:
2     def __init__(self, inhaber):
3         self.inhaber = inhaber
4         self.__guthaben = 0 # privat
5
6     def einzahlen(self, betrag):
7         if betrag > 0:
8             self.__guthaben += betrag
9
10    def guthaben_abfragen(self):
11        return self.__guthaben
12
13 k = Bankkonto("Max")
14 k.einzahlen(100)
15 print(k.guthaben_abfragen()) # 100
16 # print(k.__guthaben) # Fehler!

```

16.4 Polymorphie

```

1 class Tier:
2     def geraeusch(self):
3         pass
4
5 class Hund(Tier):
6     def geraeusch(self):
7         return "Wuff!"
8
9 class Katze(Tier):
10    def geraeusch(self):
11        return "Miau!"
12
13 for tier in [Hund(), Katze()]:
14    print(tier.geraeusch())
15 # Wuff!
16 # Miau!

```